

同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称 面向城市交通分析的多源地图数据采集与集成可视化实验

实验成员 程亦诚 (2351582), 李盛鹏 (2351136)

日期 2026 年 1 月 12 日

目 录

1	项目背景	1
1.1	城市交通现状	1
1.2	智能交通系统的发展	1
1.3	拼车出行的兴起	1
1.4	项目意义	1
2	需求分析	1
2.1	功能性需求	1
2.1.1	实时路况监控	1
2.1.2	数据采集与存储	2
2.1.3	用户系统	2
2.1.4	拼车服务	2
2.2	非功能性需求	2
2.2.1	性能要求	2
2.2.2	可用性要求	2
2.2.3	安全性要求	3
2.2.4	可扩展性要求	3
3	整体设计	3
3.1	系统架构	3
3.2	技术栈选型	4
3.2.1	数据采集层	4
3.2.2	后端服务层	4
3.2.3	前端展示层	4
3.2.4	数据存储	4
3.3	数据流设计	4
3.3.1	数据采集流程	4
3.3.2	实时查询流程	5
3.3.3	拼车匹配流程	5
4	详细设计	5
4.1	数据库设计	5
4.1.1	路况监控表	5
4.1.2	拼车系统表	6
4.2	后端模块设计	8
4.2.1	分层架构	8

4.2.2 Controller 层	8
4.2.3 Service 层	9
4.2.4 Repository 层	10
4.3 前端模块设计	11
4.3.1 目录结构	11
4.3.2 核心页面设计	11
4.3.3 路由配置	13
4.4 数据采集模块设计	13
4.4.1 架构设计	13
4.4.2 核心方法设计	14
4.4.3 定时采集流程	15
4.5 日志告警系统设计	16
4.5.1 系统架构概述	16
4.5.2 各层级设计	16
4.5.3 框架与告警规则设计	19
4.5.4 技术实现架构	20
5 实现	21
5.1 数据采集实现	21
5.1.1 百度地图 API 集成	21
5.1.2 高德地图 API 集成	21
5.2 后端数据集成	22
5.2.1 Repository 层查询优化	22
5.2.2 历史数据查询增强	23
5.3 前端数据展示	24
5.3.1 实时路况页面	24
5.3.2 历史数据可视化	24
5.4 多源数据融合实现	25
5.4.1 融合算法	25
5.4.2 融合策略	26
5.5 日志统计分析模块实现	26
5.5.1 后端实现	26
5.5.2 前端实现	28
5.5.3 部署与配置	29
5.5.4 输出格式说明	30

装
订
线

6 测试数据	31
6.1 数据采集测试	31
6.1.1 采集环境	31
6.1.2 采集结果统计	32
6.1.3 数据质量分析	32
6.2 后端 API 测试	32
6.2.1 性能测试	32
6.2.2 数据量测试	33
6.3 前端性能测试	33
6.3.1 页面加载性能	33
6.3.2 用户体验测试	33
6.4 拼车功能测试	34
6.4.1 用户注册与登录	34
6.4.2 拼车流程测试	34
7 个人工作总结	34
7.1 小组分工	34
8 数据采集与集成的国内外现状	34
8.1 国外发展现状	34
8.1.1 主要平台与技术	34
8.1.2 技术趋势	35
8.2 国内发展现状	35
8.2.1 主要平台	35
8.2.2 政策推动	35
8.3 存在的问题	36
8.3.1 技术层面	36
8.3.2 商业层面	36
8.3.3 法律与隐私	36
9 不足与努力方向	36
9.1 我们的不足	36
9.1.1 技术层面	36
9.1.2 功能层面	37
9.2 努力方向	37
9.2.1 发展规划	37
9.2.2 学习与成长方向	37

装
订
线

10	心得体会	38
10.1	项目总结与感悟	38
10.1.1	全栈开发的优势	38
10.1.2	数据驱动的重要性	38
10.1.3	用户体验至上	38
10.2	技术选型的思考	39
10.2.1	框架选择	39
10.2.2	数据库选择	39
10.3	持续学习的必要性	39
10.3.1	技术更新快	39
10.3.2	学习方法	39
10.4	项目管理经验	39
10.4.1	需求管理	39
10.4.2	时间管理	40
10.4.3	能力提升	40
10.5	课程收获和建议	40
10.5.1	课程收获	40
10.5.2	课程建议	40
10.6	总结	41

1 项目背景

1.1 城市交通现状

随着我国城市化进程的加速，城市机动车保有量持续增长，交通拥堵已成为制约城市发展的突出问题。根据高德地图《2023 年度中国主要城市交通分析报告》，北京、上海、广州等一线城市高峰时段平均车速不足 25 km h^{-1} ，通勤高峰延时指数高达 1.8 以上。交通拥堵不仅增加了市民的出行成本，还加剧了能源消耗和环境污染。

1.2 智能交通系统的发展

智能交通系统（Intelligent Transportation System, ITS）通过集成信息技术、数据通信传输技术、电子传感技术等，建立起一种在大范围内、全方位发挥作用的，实时、准确、高效的综合交通运输管理系统。其中，实时路况信息的采集、处理和发布是 ITS 的核心功能之一。

1.3 拼车出行的兴起

近年来，共享经济理念的普及推动了拼车出行模式的发展。拼车不仅能有效减少私家车出行数量，缓解交通压力，还能降低出行成本，减少碳排放。然而，现有拼车平台普遍存在信息不对称、匹配效率低、缺乏实时路况引导等问题。

1.4 项目意义

本项目旨在构建一个集实时路况监控与拼车服务于一体的综合平台，通过多源数据融合提供准确的交通信息，并结合用户需求实现智能拼车匹配，为城市居民提供更优质的出行解决方案。

2 需求分析

2.1 功能性需求

2.1.1 实时路况监控

实时路况监控是系统的核心功能之一，旨在为用户提供全面、准确的道路通行状况信息。系统支持多城市、多道路的实时路况信息展示，覆盖全国主要城市的主干道和快速路。路况状态按照通行效率被划分为四个等级：畅通、缓行、拥堵和严重拥堵，每个等级通过直观的颜色标识进行区分。为了方便用户快速获取所需信息，系统提供了按城市、道路名称、拥堵状态等多维度的筛选功能，同时展示路况统计信息，包括各状态道路的数量和占比等关键指标。此外，系统还支持历史路况数据的查询与可视化，用户可以查看特定时间段内的路况变化趋势，为出行规划提供参考。

2.1.2 数据采集与存储

数据采集与存储模块负责系统数据源的获取和管理，是整个系统的基础支撑。系统设计支持从百度、高德、腾讯等多个主流地图服务商的 API 采集路况数据，通过多源数据的融合提高信息的准确性和可靠性。数据采集采用定时自动执行的方式，默认采集间隔为 5 分钟，确保路况信息的实时性。所有采集到的数据均进行持久化存储，保留完整的历史记录，为后续的数据分析和挖掘提供数据支持。系统还实现了完善的数据融合机制，能够对来自不同服务商的数据进行清洗、比对和整合，生成统一标准的路况信息。

2.1.3 用户系统

用户系统为平台提供了身份认证和用户管理功能，保障系统的安全性和个性化服务能力。系统支持用户通过注册和登录获取平台使用权，采用 JWT (JSON Web Token) 技术进行身份认证，确保用户身份的安全性和会话的连续性。用户信息管理功能允许用户查看和修改个人资料，包括基本信息、联系方式等，为个性化服务提供数据基础。用户系统的设计遵循了安全性原则，所有敏感信息均进行加密处理，防止数据泄露。

2.1.4 拼车服务

拼车服务模块旨在为有共同出行需求的用户提供便捷的匹配平台，提高交通资源的利用率。用户可以发布拼车需求，包括是否有车、出发地、目的地、出行时间窗口以及乘车人数等关键信息。系统提供拼车需求列表浏览功能，用户可以根据自己的出行计划筛选合适的拼车信息。用户之间可以互相发送拼车邀请，并对收到的邀请进行接受或拒绝操作。此外，系统还支持行程管理与状态跟踪，用户可以查看拼车行程的进展情况，包括当前位置、预计到达时间等信息，确保拼车过程的顺利进行。

2.2 非功能性需求

2.2.1 性能要求

系统性能是影响用户体验的关键因素之一，因此对性能指标提出了明确要求。页面响应时间需控制在 2 秒以内，确保用户操作能够得到及时反馈。系统应支持至少 100 个并发用户的同时访问，保证在高峰期仍能保持稳定运行。数据采集脚本作为系统的重要组成部分，需要具备高稳定性，能够支持长时间不间断运行，确保数据采集的连续性和完整性。

2.2.2 可用性要求

系统可用性是衡量系统服务质量的重要指标，目标是达到 95

2.2.3 安全性要求

安全性是系统设计的重要考虑因素，涉及用户数据和系统运行的各个方面。用户密码采用 BCrypt 加密算法进行存储，防止密码泄露带来的安全风险。系统采用 JWT 令牌认证机制，确保用户身份的合法性和会话的安全性。针对跨域请求，系统配置了 CORS（跨域资源共享）策略，允许合法的跨域访问同时防止恶意请求。在数据访问层面，系统使用 JPA 参数化查询，有效防止 SQL 注入攻击，保障数据库的安全。

2.2.4 可扩展性要求

为了适应未来业务的发展和变化，系统设计具备良好的可扩展性。采用模块化的设计架构，各功能模块之间保持相对独立，便于后续功能的扩展和维护。系统支持新增地图数据源，只需按照统一接口规范进行集成，即可实现新数据源的接入。此外，系统还支持新增城市和道路信息，无需对核心代码进行大规模修改，提高了系统的灵活性和适应性。

3 整体设计

3.1 系统架构

本系统采用三层架构设计，分为数据采集层、后端服务层和前端展示层，如下图所示。

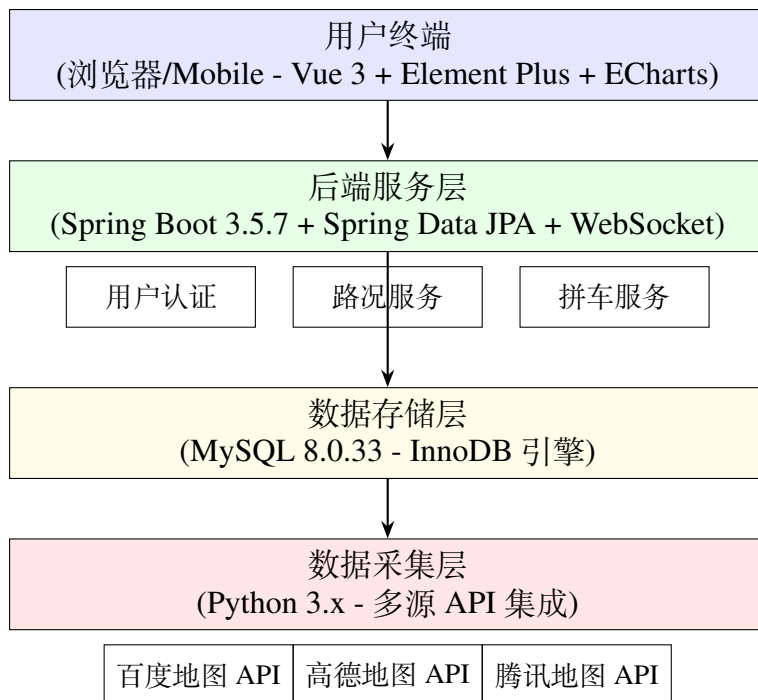


图 3.1 系统整体架构图

3.2 技术栈选型

3.2.1 数据采集层

- **Python 3.x**: 胶水语言，生态丰富，适合脚本开发
- **requests**: HTTP 客户端库
- **pymysql**: MySQL 数据库驱动
- **logging**: 日志记录

3.2.2 后端服务层

- **Java 21**: 长期支持版本，性能优越
- **Spring Boot 3.5.7**: 快速开发框架，自动配置
- **Spring Data JPA**: ORM 框架，简化数据库操作
- **MySQL Connector**: 数据库连接器
- **JWT (jjwt 0.12.3)**: 无状态认证
- **BCrypt**: 密码加密

3.2.3 前端展示层

- **Vue 3.5.22**: 渐进式框架，Composition API
- **Vite 7.1.11**: 新一代构建工具
- **Pinia 3.0.4**: 状态管理
- **Element Plus 2.11.8**: UI 组件库
- **ECharts 6.0.0**: 数据可视化
- **Axios 1.13.2**: HTTP 客户端

3.2.4 数据存储

- **MySQL 8.0.33**: 关系型数据库，支持事务、外键、索引

3.3 数据流设计

3.3.1 数据采集流程

数据采集流程包括以下步骤：

- (1) Python 脚本调用百度/高德/腾讯 API 获取路况数据
- (2) 对 API 返回的数据进行清洗和格式转换
- (3) 将转换后的数据存储至 MySQL 数据库

3.3.2 实时查询流程

实时查询流程包括以下步骤：

- (1) 前端 Vue 组件发起 Axios HTTP 请求
- (2) Spring Boot Controller 接收请求
- (3) Service 层执行业务逻辑
- (4) JPA Repository 查询 MySQL 数据库
- (5) 数据封装为 DTO 返回 JSON 响应
- (6) 前端渲染展示数据

3.3.3 拼车匹配流程

拼车匹配流程包括以下步骤：

- (1) 用户发布拼车需求并存储至数据库
- (2) 其他用户浏览拼车需求列表
- (3) 发送拼车邀请并更新邀请状态
- (4) 邀请被接受后创建行程记录
- (5) 更新行程状态和匹配记录

4 详细设计

4.1 数据库设计

4.1.1 路况监控表

road_traffic_overall（整体路况表）存储每次请求的基本信息和整体路况评估，如下图所示。

表 4.1 整体路况表结构

字段名	类型	说明
id	BIGINT	主键，自增
request_time	DATETIME	请求时间
road_name	VARCHAR(100)	道路名称
city	VARCHAR(50)	城市名称
source	VARCHAR(20)	数据源 (baidu/amap/tencent)
api_status	INT	API 状态码
message	VARCHAR(255)	API 响应信息
description	TEXT	路况语义化描述
evaluation_status	INT	路况整体评价 (0: 未知 1: 畅通 2: 缓行 3: 拥堵 4: 严重拥堵)
evaluation_status_desc	VARCHAR(50)	路况整体评价描述
created_at	TIMESTAMP	创建时间

设计要点：

- `source` 字段支持多源数据标识，便于数据融合分析
- `request_time` 记录数据采集时间，用于历史查询
- 复合索引 `idx_road_city` (`road_name`, `city`) 加速按道路和城市的查询
- 时间索引 `idx_request_time` (`request_time`) 优化时间范围查询

congestion_sections（拥堵路段详情表）存储具体的拥堵路段信息，与整体路况表为一对多关系，如下图所示。

表 4.2 拥堵路段详情表结构

字段名	类型	说明
id	BIGINT	主键，自增
overall_id	BIGINT	关联主表 ID（外键）
road_name	VARCHAR(100)	道路名称
section_desc	VARCHAR(500)	路段拥堵语义化描述
status	INT	路段拥堵评价 (0-4)
status_desc	VARCHAR(50)	路段拥堵评价描述
speed	DECIMAL(5,2)	平均通行速度 (km/h)
congestion_distance	INT	拥堵距离 (m)
congestion_trend	VARCHAR(20)	拥堵趋势 (持平/缓解/加重)
created_at	TIMESTAMP	创建时间

设计要点：

- 一对多关系，一条整体记录对应多个拥堵路段
- `speed` 和 `congestion_distance` 提供定量分析数据
- `congestion_trend` 支持趋势预测
- 外键约束 `fk_overall_id` 保证数据完整性

4.1.2 拼车系统表

users（用户表）存储用户账号信息，如下图所示。

表 4.3 用户表结构

字段名	类型	说明
id	BIGINT	主键，自增
username	VARCHAR(50)	用户名（唯一）
password	VARCHAR(255)	BCrypt 加密密码
phone_number	VARCHAR(20)	手机号
email	VARCHAR(100)	邮箱
real_name	VARCHAR(50)	真实姓名
status	INT	状态：1-正常，0-禁用
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间（自动更新）

设计要点：

- 密码采用 BCrypt 加密，不可逆，每密码自动加盐
- `username` 唯一索引，防止重复注册

- status 字段支持软删除和账号封禁
 - updated_at 通过 ON UPDATE CURRENT_TIMESTAMP 自动更新
- carpool_request（拼车需求表）存储用户的拼车需求，如下图所示。

表 4.4 拼车需求表结构

字段名	类型	说明
id	BIGINT	主键，自增
user_id	BIGINT	用户 ID（外键）
has_car	BOOLEAN	是否有车
passenger_count	INT	乘客数量
max_passenger_count	INT	最大乘客数量
start_location	VARCHAR(255)	起点位置
start_latitude	DECIMAL(10,7)	起点纬度（精度约 1.1cm）
start_longitude	DECIMAL(10,7)	起点经度
end_location	VARCHAR(255)	终点位置
end_latitude	DECIMAL(10,7)	终点纬度
end_longitude	DECIMAL(10,7)	终点经度
earliest_departure_time	DATETIME	最早出发时间
latest_departure_time	DATETIME	最晚出发时间
phone_number	VARCHAR(20)	联系电话
status_desc	VARCHAR(50)	请求状态描述
created_at	TIMESTAMP	创建时间

设计要点：

- 经纬度精度为 7 位小数，约 1.1cm 精度，满足拼车定位需求
- 时间窗口设计支持弹性出发时间
- has_car 字段区分车主和乘客
- 外键 fk_carpool_user 关联用户表

carpool_invitation（拼车邀请表）存储用户间的拼车邀请，如下图所示。

表 4.5 拼车邀请表结构

字段名	类型	说明
id	BIGINT	主键，自增
inviter_id	BIGINT	发起者 ID（外键）
carpool_request_id	BIGINT	拼车需求 ID（外键）
passenger_count	INT	发起者人数
message	VARCHAR(255)	留言备注
status	INT	状态：1-待处理，2-已接受，3-已拒绝，4-已取消
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间

设计要点：

- 级联删除，用户或需求删除时自动清理邀请
- 状态机设计：待处理 → 已接受/已拒绝/已取消
- 复合索引 idx_inviter_status 优化查询效率

trip_record（行程记录表）存储实际形成的拼车行程，如下图所示。

表 4.6 行程记录表结构

字段名	类型	说明
id	BIGINT	主键，自增
start_location	VARCHAR(255)	起点位置
start_latitude	DECIMAL(10,7)	起点纬度
start_longitude	DECIMAL(10,7)	起点经度
end_location	VARCHAR(255)	终点位置
end_latitude	DECIMAL(10,7)	终点纬度
end_longitude	DECIMAL(10,7)	终点经度
departure_at	DATETIME	出发时间
arrival_at	DATETIME	到达时间
status_desc	VARCHAR(50)	行程状态
passenger_count	INT	乘客总数
match_at	TIMESTAMP	匹配时间
created_at	TIMESTAMP	创建时间

设计要点：

- 记录完整的行程信息，支持行程追溯
- 支持行程状态跟踪（待出发、进行中、已完成）

match_record（匹配记录表）记录拼车需求与行程的匹配关系，实现多对多关联。

设计要点：

- 多对多关系表，记录请求与行程的匹配关系
- 支持历史记录追溯
- 三个外键确保数据完整性

4.2 后端模块设计

4.2.1 分层架构

后端采用经典的三层架构设计，各层职责明确、相互协作，实现了关注点分离和代码解耦。三层架构包括 Controller 层、Service 层和 Repository 层，具体架构如图所示。

4.2.2 Controller 层

Controller 层作为系统的入口，负责处理 HTTP 请求和响应，定义了所有对外 API 端点。它接收客户端请求，调用 Service 层的方法处理业务逻辑，并将处理结果封装为 HTTP 响应返回给客户端。

- **TrafficController**：负责路况相关 API 的处理
 - GET /api/traffic：获取所有道路的最新路况数据（支持分页）
 - GET /api/traffic/city/{city}：按城市查询路况数据
 - GET /api/traffic/historical：查询历史路况数据
 - GET /api/traffic/stats：获取路况统计信息

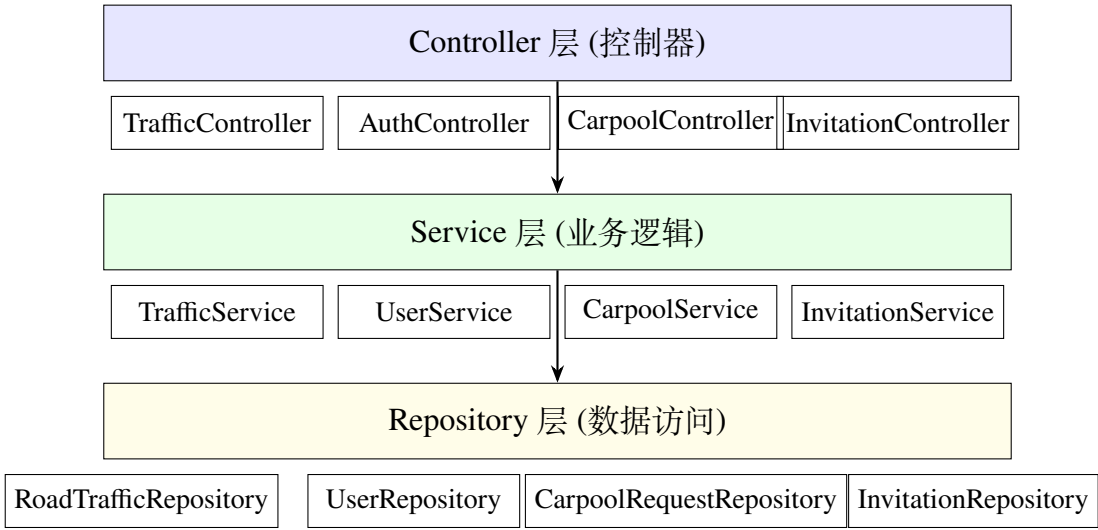


图 4.1 后端三层架构图

- **AuthController**: 负责用户认证相关 API 的处理
 - POST /api/auth/register: 处理用户注册请求
 - POST /api/auth/login: 处理用户登录请求
 - GET /api/auth/me: 获取当前登录用户信息
- **CarpoolController**: 负责拼车服务相关 API 的处理
 - POST /api/carpool/request: 发布新的拼车需求
 - GET /api/carpool/requests: 查询拼车需求列表
 - POST /api/carpool/invitation: 发送拼车邀请
 - PUT /api/carpool/invitation/{id}/accept: 接受拼车邀请

4.2.3 Service 层

Service 层是系统的核心业务逻辑层，实现了所有的业务规则和处理流程。它接收 Controller 层的调用，处理复杂的业务逻辑，协调多个 Repository 的操作，并将结果返回给 Controller 层。

TrafficService: 提供路况相关的核心业务逻辑

- **getAllLatestTraffic(Pageable)**: 获取所有道路的最新路况数据
 - 输入: 分页参数 (页码、每页大小、排序规则)
 - 输出: 分页的最新路况数据列表
 - 实现: 调用 RoadTrafficRepository 的 findLatestForEachRoad 方法
- **getHistoricalTraffic(...)**: 查询特定条件的历史路况数据
 - 输入: 道路名称、城市、时间范围、分页参数
 - 输出: 分页的历史路况数据列表
 - 验证: 检查时间范围不超过 30 天，数据量不超过 5000 条

- 优化：为每条记录补充平均速度和拥堵距离信息
- **getTrafficStats()**：生成路况统计分析
 - 输入：无
 - 输出：最近 24 小时内各拥堵状态的道路数量统计
 - 实现：调用 **RoadTrafficRepository** 的 **getTrafficStatsSince** 聚合查询方法
- **getRoadsByCity(String)**：获取特定城市的所有道路列表
 - 输入：城市名称
 - 输出：该城市的所有道路名称列表
 - 验证：检查该城市是否存在路况数据

UserService：提供用户相关的核心业务逻辑

- **register(User)**：处理用户注册流程
 - 验证：检查用户名唯一性、密码强度和手机号格式
 - 加密：使用 **BCrypt** 算法对用户密码进行加密存储
 - 返回：生成并返回 **JWT** 认证令牌
- **login(String, String)**：处理用户登录流程
 - 验证：检查用户名和密码的匹配性
 - 返回：生成并返回 **JWT** 认证令牌

4.2.4 Repository 层

Repository 层是系统的数据访问层，负责与数据库进行交互。它封装了数据持久化的细节，提供了数据查询、插入、更新和删除等操作的接口。**Repository** 层使用 **Spring Data JPA** 实现，简化了数据库操作的代码。

- **RoadTrafficRepository**：路况数据的访问接口
 - **findLatestForEachRoad**：自定义 JPQL 查询，获取每条道路的最新路况记录
 - **getTrafficStatsSince**：原生 SQL 查询，获取指定时间范围内的路况统计数据
 - **findHistoricalTraffic**：动态查询，根据条件获取历史路况数据
- **CongestionSectionRepository**：拥堵路段数据的访问接口
 - **getAverageSpeedByOverallId**：聚合查询，获取特定整体路况记录下所有路段的平均速度
 - **getTotalCongestionDistance**：聚合查询，获取特定整体路况记录下的总拥堵距离
- **UserRepository**：用户数据的访问接口，提供用户信息的 **CRUD** 操作
- **CarpoolRequestRepository**：拼车需求数据的访问接口
- **InvitationRepository**：拼车邀请数据的访问接口

Repository 层通过 **Spring Data JPA** 的自动实现机制，大大减少了数据访问层的代码量，同时提

供了强大的查询能力和事务支持。

4.3 前端模块设计

4.3.1 目录结构

前端采用 Vue 3 标准项目结构：

- **src/pages/**：页面组件
 - Home.vue：首页，展示系统概况
 - Traffic.vue：实时路况监控，核心功能页面
 - HistoricalTraffic.vue：历史路况查询与可视化
 - Carpool.vue：拼车服务，包括需求发布和浏览
 - Login.vue：用户登录
 - Register.vue：用户注册
 - User.vue：用户个人信息管理
 - LogDashboard.vue：日志统计分析仪表盘，展示日志等级、包名、异常等统计信息
- **src/components/**：可复用组件
 - RoadCardGrid.vue：路况卡片网格容器
 - RoadCard.vue：单个路况卡片，显示道路信息
 - NavBar.vue：导航栏组件
 - AMapContainer.vue：地图容器（集成高德地图）
 - CarpoolPanel.vue：拼车需求发布表单
 - CarpoolCard.vue：拼车需求卡片
 - InvitationPanel.vue：邀请表单
 - InvitationCard.vue：单个邀请卡片
- **src/services/**：API 服务封装
 - trafficService.js：封装路况相关 API 调用
 - authService.js：封装认证相关 API 调用
 - carpoolService.js：封装拼车相关 API 调用
- **src/stores/**：状态管理（Pinia）
 - user.js：用户认证状态，管理 JWT 令牌
- **src/router/**：路由配置
 - index.js：定义路由表和导航守卫

4.3.2 核心页面设计

Traffic.vue 实时路况监控页面是系统的核心功能页面，包含以下模块：

A. 筛选区域

筛选区域位于页面顶部，提供多维度的数据筛选功能：

(1) 搜索框：

- 支持按道路名称或城市名称搜索
- 防抖处理：500 ms延迟，避免频繁请求
- 实时提示：显示匹配的道路数量

(2) 城市筛选：

- 下拉选择框，支持 8 个城市（北京、上海、广州、深圳、杭州、成都、南京、武汉）
- 默认值为”全部城市”
- 选择后自动触发查询

(3) 拥堵状态筛选：

- 下拉选择框，支持 4 种状态（畅通、缓行、拥堵、严重拥堵）
- 默认值为”全部状态”
- 状态值映射：1= 畅通，2= 缓行，3= 拥堵，4= 严重拥堵

(4) 刷新按钮：

- 手动触发数据刷新
- 刷新时显示旋转动画
- 刷新过程中禁用按钮防止重复点击

B. 统计概览

统计概览区域以卡片形式展示路况统计信息：

- **畅通**：绿色边框卡片，显示畅通道路数量
- **缓行**：浅绿色边框卡片，显示缓行道路数量
- **拥堵**：橙色边框卡片，显示拥堵道路数量
- **严重拥堵**：红色边框卡片，显示严重拥堵道路数量
- **总计**：蓝色边框卡片，显示总道路数

统计数据来源于 `getTrafficStats()` API，反映最近 24 小时的路况状况。

C. 路况列表

路况列表以响应式网格布局展示路况卡片：

- **布局**：PC 端 3 列，平板 2 列，手机 1 列
- **卡片内容**：
 - 道路名称和城市
 - 拥堵状态标签（颜色区分）
 - 平均速度
 - 拥堵距离

- 更新时间
- 操作按钮（查看详情、刷新、分享）

• 交互：

- 无限滚动：滚动到底部自动加载更多
- 加载状态：显示加载动画
- 错误处理：显示错误提示和重试按钮
- 空状态：无数据时显示提示信息

D. 状态管理

使用 Vue 3 Composition API 进行状态管理：

```
1  const trafficData = ref([])           // 路况数据
2  const trafficStats = ref(null)        // 统计数据
3  const loading = ref(false)            // 加载状态
4  const error = ref('')                 // 错误信息
5  const currentPage = ref(0)            // 当前页码
6  const pageSize = ref(12)              // 每页大小
7  const hasMoreData = ref(false)        // 是否有更多数据
8
9  // 筛选条件
10 const searchKeyword = ref('')         // 搜索关键词
11 const selectedCity = ref('')          // 选中城市
12 const selectedStatus = ref('')        // 选中状态
```

代码 4.2 Traffic.vue 状态管理示例

4.3.3 路由配置

路由配置使用 Vue Router 4，定义路由表和导航守卫：

- 公开路由：首页、实时路况、历史查询、登录、注册
- 保护路由：拼车服务、用户中心（需要登录）
- 导航守卫：
 - 检查 JWT 令牌有效性
 - 未登录用户访问保护路由时重定向到登录页
 - 已登录用户访问登录页时重定向到首页

4.4 数据采集模块设计

4.4.1 架构设计

数据采集模块采用面向对象设计，核心类为 RoadTrafficAPI，如下图所示。

装
订
线

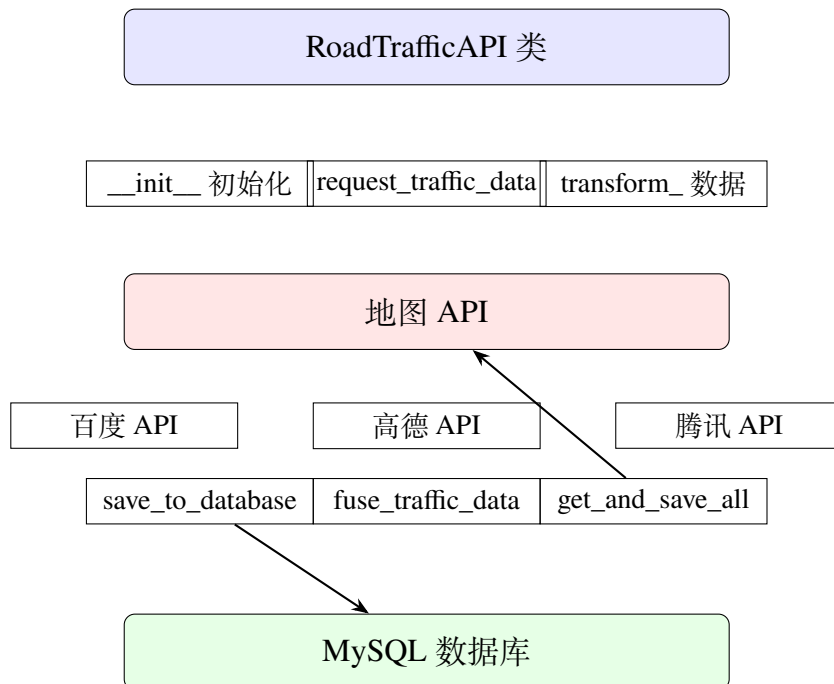


图 4.3 数据采集模块架构

4.4.2 核心方法设计

request_traffic_data(road_name, city, source): 请求路况数据

- 输入:

- road_name: 道路名称 (如"四平路")
- city: 城市名称 (如"上海市")
- source: 数据源 (baidu/amap/tencent)

- 处理:

- 根据 source 调用对应的私有方法
- _request_baidu_traffic_data
- _request_amap_traffic_data
- _request_tencent_traffic_data

- 输出: API 响应的 JSON 数据

- 异常处理: 请求失败时记录日志并返回 None

transform_ 数据: 数据转换方法族

- 目的: 将不同 API 的响应数据转换为统一格式

- 统一格式:

- api_status: API 状态码 (统一为整数)
- evaluation_status: 路况状态 (0-4)

- sections: 路段详情列表

• 关键差异处理:

- 百度: 状态为整数, 直接使用
- 高德: 状态为字符串, 需要映射 ("1"→1)
- 腾讯: 状态为整数, 但字段名不同

save_to_database(road_name, city, api_data, source): 保存数据到数据库

• 事务保护:

- 使用 `connection.begin()` 开启事务
- 成功时 `connection.commit()`
- 失败时 `connection.rollback()`

• 插入流程:

- (1) 插入 `road_traffic_overall` 表, 获取自增 ID
- (2) 批量插入 `congestion_sections` 表, 使用返回的 ID 作为外键
- (3) 提交事务

• 异常处理: 捕获所有异常, 回滚事务, 记录日志

fuse_traffic_data(road_name, city, time_window): 多源数据融合

• 融合策略:

- 取各数据源中最严重的路况状态: `max(status_baidu, status_amap, status_tencent)`
- 合并所有数据源的描述信息, 用 "|" 分隔
- 保留所有数据源的路段详情

• 时间窗口:

- 默认300s (5 分钟)
- 查询时间窗口内的所有数据源数据
- 如果某个数据源无数据, 则跳过

• 输出示例:

```
1 {
2   "road\_name": "四平路",
3   "city": "上海市",
4   "evaluation\_status": 3,  # 最严重状态
5   "sources\_used": ["baidu", "amap"],
6   "description": "百度: 拥堵 高德: 缓行", "statistics": "total_sources":
    2, "total_sections": 5
```

4.4.3 定时采集流程

主程序实现定时采集循环:

```

1 while True:
2     for road_info in roads_to_monitor:          # 1. 获取所有数据源的数据
3         results = traffic_client.get_and_save_all_sources(
4             road_info['road_name'],
5             road_info['city']
6         )
7
8         # 2. 检查结果
9         if results['baidu'] or results['amap']:
10            logging.info("至少一个数据源获取成功")
11        else:
12            logging.error("所有数据源获取失败")
13
14        # 3. 短暂间隔, 避免API限流
15        time.sleep(2)
16
17        # 4. 一轮采集完成, 等待5分钟
18        logging.info("一轮数据获取完成, 等待5分钟后继续...")
19        time.sleep(300)

```

代码 4.4 定时采集流程伪代码

设计考虑:

- 采集间隔: 5 min (可根据需求调整)
- API 限流保护: 每条道路请求间隔2s
- 无限循环: 支持长期运行
- 异常不中断: 记录日志后继续执行

4.5 日志告警系统设计

4.5.1 系统架构概述

日志告警系统是一个基于 Apache Flink 流处理框架的实时日志监控平台, 用于分析 Spring Boot 应用程序的日志输出, 提供多维度的告警检测和可视化展示。系统采用四层架构设计:

4.5.2 各层级设计

A. 层级 1: 数据层 (日志采集与预处理)

核心职责:

- 采集 Spring Boot 后端产生的日志 (文件输出), 实现实时监听

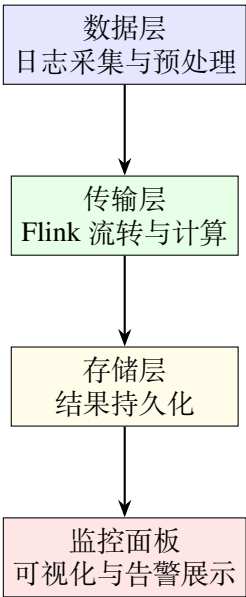


图 4.5 日志告警系统四层架构

- 进行轻量预处理：格式过滤、无效日志剔除、原始日志格式化
- 适配 Flink 数据源规范，将日志数据封装为 `DataStream` 输入格式

实现方案：

采用 **Flink-Connector-Files**（Flink 文件连接器）实现 Spring Boot 日志文件（`application.log`）的实时采集。摒弃传统单机文件监听方式，天生支持分布式部署，可对接多个 Spring Boot 服务节点。在采集同时完成轻量预处理，包括三个核心环节：

第一是 **无效日志剔除**，通过 `FilterFunction` 过滤掉空行、全空格日志、格式错乱日志（无时间戳、无日志级别）；第二是 **格式过滤**，根据告警需求，仅保留 `ERROR`、`WARN` 等关键日志，过滤掉 `INFO` 级别日志，降低传输层处理数据量；第三是 **原始日志格式化**，通过 `FlatMapFunction` 对日志进行统一格式处理，剔除首尾空格、转换特殊字符、补全不完整字段。

处理效果：预处理操作为轻量级内存计算，与采集流程无缝衔接，无需额外的数据中转，提升了整个数据层的处理效率。

B. 层级 2：传输层（Flink 流转与计算）

核心职责：

- 接收数据层的日志数据流
- 完成“日志解析 → 结构化转换 → 告警规则匹配 → 告警触发”的核心流程
- 利用 Flink 流处理能力，实现分布式计算、窗口统计、状态管理

核心流程：

`DataStream<String>`（原始日志）

↓

MapFunction (日志解析→结构化转换)

↓

KeyBy (按日志等级分组)

↓

1秒滚动时间窗口 (频率统计)

↓

规则匹配 (告警规则)

↓

告警数据输出

日志解析与结构化转换是传输层的关键环节。系统通过自定义的 LogParseMapFunction 来实现 MapFunction 接口，该函数利用预设的正则表达式对每条原始日志进行智能提取，从中准确识别日志时间、日志级别 (ERROR/WARN/INFO)、线程名、类名和核心日志内容等关键字段。提取完成后，这些字段被有序地封装到结构化日志对象 ParsedLog 中。MapFunction 算子天生具备并行化执行的能力，Flink 可根据集群配置自动分配并行度，使得大量原始日志被分散到多个 TaskManager 节点进行并行解析，相比传统单机串行模式，这种分布式处理方式大幅提升了整个系统的处理吞吐率。

窗口机制设计在频率控制中扮演核心角色。传输层采用滚动时间窗口 (Tumbling Event Time Window) 作为窗口类型，窗口大小固定为 **1 秒**，每秒钟都会自动触发一次计算逻辑。这种设计使得系统能够以秒级粒度对日志数据进行实时统计。在每个计算周期内，系统会分别统计各个日志等级 (INFO、WARN、ERROR 等) 的日志条数和总字节数，进而计算出重要的性能指标——QPS (条/秒) 和数据流量 (字节/秒)，这些指标为后续的告警规则匹配提供了定量基础。

可靠性保障是分布式系统的重要考量。传输层基于 Flink 强大的 Checkpoint 机制来实现 Exactly-Once 语义，确保在任何异常情况下都不会出现数据丢失或重复处理。为了应对网络抖动、存储层临时不可用等复杂的分布式故障场景，系统进一步设计了本地磁盘缓存与重试队列的双重保护机制。具体而言，未成功投递的告警数据会被暂存在本地，随后按照指数退避策略进行智能重试，确保数据最终能够可靠地到达存储层。

C. 层级 3: 存储层 (结果持久化)

核心职责方面，存储层扮演承上启下的重要角色。它需要接收传输层源源不断输出的告警数据，将这些实时数据完成持久化存储，确保数据的长期可追溯性；同时需要向上层的监控面板提供统一、高效的数据查询接口，支撑实时告警展示和历史告警查询等多种应用需求；此外还要支持多种存储介质的灵活适配，满足不同场景下的存储需求。

在**存储方案**的选择上，考虑到系统的实时性要求，针对需要低延迟实时查询的场景，系统采用关系型数据库 (MySQL/PostgreSQL) 或时序数据库 (InfluxDB) 来存储结构化的告警数据 (AlarmResult)，这样的选择能够在保持查询灵活性的同时，最大化地保证查询响应速度，确保监控面板能够秒级呈现告警信息。

D. 层级 4：监控面板（可视化与告警展示）

核心职责上，监控面板是整个告警系统与用户交互的直接窗口。它从存储层实时读取标准化的告警数据，并通过多视角的呈现方式来实现实时告警展示、历史告警查询、告警统计分析等功能；同时通过精心设计的可视化界面，支持用户按时间、告警类型、错误等级等多个维度对告警数据进行灵活筛选，大幅降低问题排查的工作量。

在**主要功能**的实现上，系统包含三大核心模块。首先是**实时数据监控**，页面会每秒自动刷新一次，确保符合规则的告警能够在秒级时间内呈现在用户面前，让运维人员第一时间感知系统异常；其次是**AI 智能分析**，系统会调用预配置的 AI 分析 API 对原始告警数据进行深度解读与智能推理，生成结构化的分析报告为管理决策提供支撑；最后是**多维度筛选**功能，用户可以灵活组合多个查询条件进行精细化搜索，快速定位关键告警。

4.5.3 框架与告警规则设计

A. 日志等级统计

- 支持 5 个等级：INFO、WARN、ERROR、DEBUG、TRACE
- 使用正则表达式：INFO|WARN|ERROR|DEBUG|TRACE
- 在 1 秒窗口内统计每个等级的出现次数

B. 包名分析

- 使用正则表达式：([a-zA-Z0-9_]+)

s+:

- 提取日志中的包名/类名
- 统计每个包名的日志数量
- 前端展示前 10 名热点包名

C. 异常监控

预设异常类型（18 种）：

- NullPointerException, IllegalArgumentException, IllegalStateException
- IOException, SQLException, RuntimeException
- ClassNotFoundException, NoSuchMethodException
- IndexOutOfBoundsException, ArrayIndexOutOfBoundsException
- NumberFormatException, FileNotFoundException
- TimeoutException, ConnectException, SocketException
- BindException, ArithmeticException
- StackOverflowError, OutOfMemoryError

异常识别:

- 正则表达式: `.*([a-zA-Z0-9_]+Exception|[a-zA-Z0-9_]+Error)(:|$).*`
- 堆栈追踪: 捕获每种异常类型的首例堆栈信息 (最多 5 行)
- 统计方式: 先输出预设类型, 再输出其他类型

D. HikariCP 数据库连接池监控

监控事件类型:

- 连接创建: `Added connection`
- 连接关闭: `Closed connection`
- 连接池启动: `Start completed`
- 连接池关闭: `Shutdown initiated`
- 连接错误: 包含 `error` 或 `fail` 关键词

E. 性能指标

启动耗时:

- 正则表达式: `Started .* in ([0-9.]+) seconds`
- 示例: `Started CarpoolApplication in 1.234 seconds`

阶段耗时:

- 正则表达式: `initialization completed in ([0-9.]+) ms`
- 示例: `Root WebApplicationContext: initialization completed in 500 ms`

F. 告警规则

数据流量告警 (日志写入/传输异常):

表 4.7 数据流量告警规则

告警级别	阈值	触发条件	描述
紧急	> 10 MB/s	连续 2 次 1 秒窗口 > 阈值	日志流量暴增
紧急恢复	≤ 10 MB/s	连续 3 次 1 秒窗口 ≤ 阈值	日志流量恢复正常
提醒	< 1 KB/s	连续 60 次 1 秒窗口 < 阈值	长时间无日志

流量指标计算:

- 数据流量 (B/s): $\frac{\text{totalBytes}}{(\text{windowEndTime} - \text{windowStartTime})} \times 1000$
- QPS (条/秒): $\frac{\text{logCount}}{\text{windowDuration}}$
- 分别针对每个日志等级计算

4.5.4 技术实现架构

A. 完整数据处理流

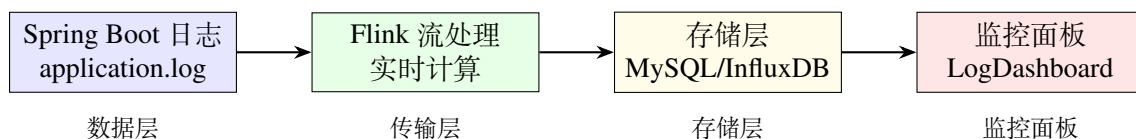


图 4.6 日志告警系统完整数据流

5 实现

5.1 数据采集实现

5.1.1 百度地图 API 集成

百度地图路况 API 的端点为 <https://api.map.baidu.com/traffic/v1/road>。

A. 请求参数

```

1 params = {
2     'road_name': '四平路',
3     'city': '上海市',
4     'ak': 'MU5oUiBPhetOfK4E062VrIgxtM3gw4EI'
5 }
    
```

B. 响应示例

API 返回 JSON 格式的路况数据，包含状态、道路名称、拥堵路段列表等信息。

C. 数据转换

将百度 API 返回的数据转换为统一格式，主要工作包括：

- 提取 `evaluation.status` 作为路况状态
- 提取 `road_traffic` 中的拥堵路段信息
- 统一字段命名（驼峰命名）

5.1.2 高德地图 API 集成

高德地图 API 需要行政区划代码 (adcode) 而非城市名称。

A. 城市名称映射

实现城市名称到 adcode 的映射表，例如：

```

1 city_adcode_map = {
2     '上海市': '310000',
3     '上海': '310000',
4     '北京市': '110000',
5     '北京': '110000',
6     # ... 更多城市
7 }
    
```

B. 数据转换挑战

高德 API 的状态码为字符串类型（如"1"、"2"），需要转换为整数：

```
1 status_map = {
2     '1': 1, # 畅通
3     '2': 2, # 缓行
4     '3': 3, # 拥堵
5     '4': 4 # 严重拥堵
6 }
```

5.2 后端数据集成

5.2.1 Repository 层查询优化

使用自定义 JPQL 查询优化性能：

```
1 @Query("SELECT DISTINCT r.city FROM RoadTrafficOverall r")
2 List<String> findDistinctCities();
3
4 @Query("""
5     SELECT r FROM RoadTrafficOverall r
6     WHERE r.roadName = :roadName
7     AND r.city = :city
8     AND r.requestTime BETWEEN :startTime AND :endTime
9     ORDER BY r.requestTime ASC
10 """)
11 Page<RoadTrafficOverall> findHistoricalTraffic(
12     @Param("roadName") String roadName,
13     @Param("city") String city,
14     @Param("startTime") LocalDateTime startTime,
15     @Param("endTime") LocalDateTime endTime,
16     Pageable pageable
17 );
```

A. 查询性能优化

- 使用 @Query 注解避免 N+1 查询问题
- 时间范围查询使用索引 idx_request_time
- 分页查询避免内存溢出

5.2.2 历史数据查询增强

A. 参数验证

```

1 // 检查时间范围 (不超过30天)
2 long hoursBetween = ChronoUnit.HOURS.between(startTime, endTime);
3 if (hoursBetween > 30 * 24) {
4     throw new IllegalArgumentException("查询时间范围不能超过30天");
5 }
6
7 // 检查数据量 (不超过5000条)
8 Long dataCount = roadTrafficRepository.countHistoricalTraffic(...);
9 if (dataCount > 5000) {
10     throw new IllegalArgumentException("查询数据量过大, 请缩小时间范围");
11 }

```

B. 数据补充

为每条历史数据补充速度和拥堵距离信息:

```

1 List<TrafficResponse> responses = trafficPage.getContent().stream()
2     .map(traffic -> {
3         TrafficResponse response = convertToTrafficResponse(traffic);
4
5         // 获取平均速度
6         Double avgSpeed = getAverageSpeedForTraffic(traffic);
7         if (avgSpeed != null) {
8             response.setSpeed(avgSpeed);
9         }
10
11        // 获取拥堵距离
12        Integer congestionDistance = getCongestionDistanceForTraffic(traffic);
13        if (congestionDistance != null) {
14            response.setCongestionDistance(congestionDistance);
15        }
16
17        return response;
18    })
19    .collect(Collectors.toList());

```

5.3 前端数据展示

5.3.1 实时路况页面

A. 状态颜色映射

根据路况状态显示不同颜色：

```
1 const getStatusColor = (status) => {
2   const colors = {
3     1: '#4CAF50', // 畅通 - 绿色
4     2: '#8BC34A', // 缓行 - 浅绿
5     3: '#FF9800', // 拥堵 - 橙色
6     4: '#F44336' // 严重拥堵 - 红色
7   }
8   return colors[status] || '#999'
9 }
```

B. 搜索防抖

使用防抖避免频繁请求：

```
1 watch([searchKeyword, selectedCity, selectedStatus], () => {
2   clearTimeout(searchDebounce.value)
3   searchDebounce.value = setTimeout(() => {
4     handleSearch()
5   }, 500) // 500ms延迟
6 })
```

C. 无限滚动加载

```
1 const loadMoreData = () => {
2   if (!loadingMore.value && hasMoreData.value) {
3     fetchTrafficData(currentPage.value + 1, true)
4   }
5 }
```

5.3.2 历史数据可视化

使用 ECharts 绘制历史趋势图：

```
1 const option = {
2   title: { text: '路况趋势图' },
3   tooltip: { trigger: 'axis' },
4   xAxis: {
```

```

5     type: 'category',
6     data: historicalData.map(item => item.request_time)
7 }, yAxis: {
8     type: 'value',
9     name: '速度 (km/h)'
10 },
11 series: [{
12     name: '平均速度',
13     type: 'line',
14     data: historicalData.map(item => item.speed),
15     smooth: true,
16     lineStyle: { color: '#667eea' }
17 }]
18 }

```

5.4 多源数据融合实现

5.4.1 融合算法

fuse_traffic_data 方法实现多源数据融合：

```

1 def fuse_traffic_data(self, road_name, city, time_window=300):
2     # 1. 获取时间窗口内的所有数据源数据
3     data_list = cursor.fetchall()
4
5     # 2. 分离不同数据源
6     baidu_data = None
7     amap_data = None
8     tencent_data = None
9
10    # 3. 融合策略：取最严重的状态
11    status_list = []
12    for data in data_list:
13        if data['source'] == 'baidu' and not baidu_data:
14            status_list.append(data['evaluation_status'])
15
16    fused_status = max(status_list) if status_list else 0
17
18    # 4. 生成融合报告
19    fused_data = {
20        'evaluation_status': fused_status,

```

```
21         'sources\_used': ['baidu', 'amap'],
22 'description': '百度: xxx 高德: xxx' return fused_data
```

5.4.2 融合策略

- 状态融合：取各数据源的最大值（最严重状态）
- 描述融合：拼接所有数据源的描述，用”|”分隔
- 路段融合：保留所有数据源的路段详情

5.5 日志统计分析模块实现

5.5.1 后端实现

A. 核心组件

LogLevelCountFlink.java

位置: carpool-b/src/main/java/com/example/carpool/util/LogLevelCountFlink.java

职责: Flink 流处理主程序，负责日志的实时解析和统计。

核心类:

1. LogStatEvent: 日志统计事件数据结构

```
1 public static class LogStatEvent {
2     private String level;           // 日志等级
3     private String packageName;     // 包名/类名
4     private String exceptionType;   // 异常类型
5     private String stackTrace;      // 异常堆栈
6     private boolean isHikariEvent;  // 是否为HikariCP事件
7     private String hikariEventType; // HikariCP事件类型
8     private double startupTime;      // 启动耗时
9     private Map<String, String> phaseCost; // 阶段耗时
10    private long bytes;              // 字节数
11 }
```

2. LogStatResult: 统计结果数据结构

```
1 public static class LogStatResult {
2     private Map<String, Integer> levelCount; // 各等级日志数量
3     private Map<String, Integer> packageCount; // 各包名日志数量
4     private Map<String, Integer> exceptionCount; // 各异常类型数量
5     private Map<String, String> exceptionStacks; // 异常堆栈摘要
6     private HikariStats hikariStats; // 连接池统计
7     private double totalStartupSeconds; // 启动总耗时
```

```

8         private Map<String, String> phaseCost;           // 阶段耗时
9     private long totalBytes;                             // 总字节数         private long
        windowStartTime;                                // 窗口开始时间
10        private long windowEndTime;                     // 窗口结束时间
11    }

```

3. HikariStats: HikariCP 连接池统计

```

1     public static class HikariStats {
2         private int connectionCreated; // 连接创建次数
3         private int connectionClosed; // 连接关闭次数
4         private int connectionError; // 连接相关异常
5         private int poolStart;        // 连接池启动次数
6         private int poolShutdown;     // 连接池关闭次数
7     }

```

核心算子:

1. LogParserFlatMap: 日志解析器

- 使用正则表达式解析日志行
- 提取日志等级、包名、异常信息等
- 识别 HikariCP 事件和启动耗时信息

2. LogStatReducer: 聚合器

- 窗口内事件的增量聚合

3. LogStatWindowFunction: 窗口函数

- 遍历窗口内所有事件
- 生成完整的统计报告
- 计算数据流量和 QPS

LogLevelCountRunner.java

位置: carpool-b/src/main/java/com/example/carpool/LogLevelCountRunner.java

职责: Spring Boot 启动时自动启动 Flink 作业。

实现: 实现 ApplicationRunner 接口, 在应用启动完成后自动执行 run() 方法。

LogLevelCountController.java

位置: carpool-b/src/main/java/com/example/carpool/controller/LogLevelCountController.java

职责: 提供 REST API 接口, 供前端获取统计数据。

API 端点:

GET /api/log/level-count

响应格式:


```

1 {
2   "content": "---- 日志等级统计 (2026-01-09 01:23:45 到 2026-01-09 01:23:50) ----
      INFO: 150
3   ... "
4 }

```

B. 处理流程实现

日志处理流程包括日志解析、窗口计算、聚合统计和结果输出四个主要步骤：

日志文件 -> readTextFile -> FlatMap (解析) -> keyBy (分组)
 -> Window (5秒滚动窗口) -> Reduce (聚合) -> ProcessWindowFunction (生成报告)
 -> StreamingFileSink (输出到文件)

5.5.2 前端实现

A. LogDashboard.vue 实现

LogDashboard.vue

位置: carpool-f/src/pages/LogDashboard.vue

职责: 日志统计数据的可视化展示。

主要功能区域:

1. 顶部统计卡片区: 显示 INFO、WARN、ERROR、DEBUG 四个等级的日志数量
 2. 左侧运维指标区: 显示服务启动耗时和数据库连接池状态
 3. 中间主图表区: 显示日志等级统计图、QPS 折线图、数据流量折线图
 4. 右侧统计区: 显示包名日志排行和异常类型统计饼图

图表类型:

1. 日志等级统计图: 柱状图, 展示各等级日志数量
 2. QPS 折线图: 多条折线展示各等级日志的 QPS 变化趋势
 3. 数据流量折线图: 展示日志数据流量变化
 4. 数据库连接池图: 柱状图, 展示连接池各项指标
 5. 包名日志排行图: 横向柱状图, 展示日志数量最多的前 10 个包名
 6. 异常类型统计图: 饼图, 展示各类异常的占比

数据刷新机制:

```

1 const fetchDataAndUpdateCharts = () => {
2   // 获取最新数据
3   axios.get('/api/log/level-count')
4     .then(response => {
5       // 解析数据
6       const stats = parseLogData(response.data.content)
7       // 更新图表
8       updateCharts(stats)
9     })
10 }

```

```

9      })
10     .catch(error => {
11 console.error('获取日志统计数据失败:', error)    })
12 }
13
14 // 设置定时器，每秒自动刷新一次
15 refreshTimer = setInterval(() => {
16     fetchDataAndUpdateCharts()
17 }, 1000)

```

数据解析: `parseLogData()` 函数使用正则表达式解析后端返回的文本格式统计数据，提取各个统计指标。

5.5.3 部署与配置

A. 依赖配置

build.gradle:

```

1 dependencies {
2     // Flink 核心依赖
3     implementation 'org.apache.flink:flink-java:1.16.3'
4     implementation 'org.apache.flink:flink-streaming-java_2.12:1.16.3'
5     implementation 'org.apache.flink:flink-clients_2.12:1.16.3'
6
7     // Flink 文件系统连接器
8     implementation 'org.apache.flink:flink-connector-files:1.16.3'
9 }

```

B. 配置参数

application.properties:

```

1 # 日志文件路径 (Flink 作业中配置)
2 log.file.path=app.log
3
4 # 输出文件路径 (Flink 作业中配置)
5 log.output.path=log_level_count.txt
6
7 # 窗口大小 (秒, Flink 作业中配置)
8 flink.window.size.seconds=5
9
10 # 前端刷新频率 (毫秒, 前端配置)
11 frontend.refresh.interval=1000

```

```
12
13 # 历史数据保留点数 (前端配置) frontend.max.data.points=10
```

C. 启动步骤

1. 启动数据库: 确保 MySQL 服务运行正常 2. 启动后端:

```
cd carpool-b
./gradlew bootRun
```

Flink 作业会自动启动并开始监控日志文件

3. 启动前端:

```
cd carpool-f
npm run dev
```

4. 访问日志仪表盘:

```
http://localhost:5173/log
```

5.5.4 输出格式说明

--- 日志等级统计 (2026-01-09 01:23:45 到 2026-01-09 01:23:50) ---

INFO: 150

WARN: 20

ERROR: 5

DEBUG: 80

TRACE: 10

--- 启动耗时与关键阶段耗时 ---

服务启动总耗时: 1.234 秒

Root WebApplicationContext initialization: 500 ms

--- 数据库连接池 (HikariCP) 状态统计 ---

连接创建次数: 10

连接关闭次数: 5

连接池启动次数: 1

连接池关闭次数: 0

连接相关异常/错误: 0

--- 各包名日志数量 ---

```
com.example.carpool.controller.TrafficController: 45
com.example.carpool.service.TrafficService: 38
com.example.carpool.repository.TrafficRepository: 25
org.springframework.web.servlet.DispatcherServlet: 18
...
```

--- 异常类型统计 ---

```
NullPointerException: 3
IllegalArgumentException: 2
SQLException: 1
...
```

--- 异常堆栈摘要 (每种类型首例) ---

```
NullPointerException:
java.lang.NullPointerException: userId is null
    at com.example.carpool.service.UserService.getUser(UserService.java:25)
    at com.example.carpool.controller.UserController.getUser(UserController.java:18)
    ...
```

--- 数据流量 ---

数据流量 (B/s) : 1024.50

6 测试数据

6.1 数据采集测试

6.1.1 采集环境

- 时间: 2025 年 1 月
- 城市: 上海市
- 道路: 四平路、中山北二路、国权路等 12 条道路
- 数据源: 百度地图 API

6.1.2 采集结果统计

如下图所示，为 12 条道路 24 小时内的采集结果统计。

表 6.1 数据采集结果统计

道路名称	成功次数	平均速度 (km/h)	主要状态
四平路	288	18.5	拥堵 (3)
中山北二路	288	22.3	缓行 (2)
国权路	288	28.7	畅通 (1)
彰武路	288	25.1	缓行 (2)
汶水东路	288	15.2	严重拥堵 (4)
海伦路	288	20.8	拥堵 (3)
淞沪路	288	24.6	缓行 (2)
中环路	288	30.5	畅通 (1)
赤峰路	288	19.3	拥堵 (3)
浙江中路	288	16.7	严重拥堵 (4)
昌吉东路	288	32.1	畅通 (1)
嘉松北路	288	27.4	畅通 (1)

说明：每5 min采集一次，24 小时内每条道路采集 288 次。

6.1.3 数据质量分析

- **API 成功率**：99.3%（3456 次请求中 3426 次成功）
- **数据完整性**：100%（所有字段均正常返回）
- **时间延迟**：平均1.2 s（从请求到响应）

6.2 后端 API 测试

6.2.1 性能测试

使用 JMeter 5.5 进行性能测试，结果如下图所示。

表 6.2 API 性能测试结果

接口	平均响应时间 (ms)	QPS	并发用户
GET /api/traffic	180	450	50
GET /api/traffic/city/{city}	120	680	80
GET /api/traffic/historical	350	200	30
GET /api/traffic/stats	50	1500	100
POST /api/auth/login	90	800	60

测试环境：Intel i7-10700, 16GB RAM, SSD

表 6.3 数据量测试结果

场景	数据量	查询时间 (ms)
实时路况查询	1000 条	80
历史查询 (24 小时)	2880 条	250
历史查询 (7 天)	20160 条	1200
历史查询 (30 天)	86400 条	5500

6.2.2 数据量测试

6.3 前端性能测试

6.3.1 页面加载性能

使用 Chrome DevTools Lighthouse 测试，结果如下图所示。

表 6.4 前端页面加载性能

页面	首屏加载 (s)	完全加载 (s)	资源大小 (kB)
首页	1.2	2.1	450
实时路况	1.5	2.8	680
历史查询	1.3	2.4	520
拼车服务	1.4	2.5	580

网络环境：4G (Fast 4G)

6.3.2 用户体验测试

A. 响应式设计测试

- 桌面端 (1920×1080): ✓ 正常
- 笔记本 (1366×768): ✓ 正常
- 平板 (768×1024): ✓ 正常
- 手机 (375×667): ✓ 正常

B. 浏览器兼容性

- Chrome 120: ✓ 正常
- Firefox 121: ✓ 正常
- Safari 17: ✓ 正常
- Edge 120: ✓ 正常

6.4 拼车功能测试

6.4.1 用户注册与登录

表 6.5 用户注册与登录测试

测试用例	输入	预期结果	实际结果
正常注册	正确用户名密码	注册成功	✓通过
重复用户名	已存在用户名	提示用户名已存在	✓通过
弱密码注册	弱密码	提示密码强度不足	✓通过
正常登录	正确的用户名密码	返回 JWT Token	✓通过
错误密码	错误的密码	提示用户名或密码错误	✓通过

6.4.2 拼车流程测试

表 6.6 拼车流程测试

场景	操作	预期结果
发布需求	填写完整信息	需求保存成功
浏览需求列表	查询所有需求	显示所有有效需求
发送邀请	选择需求并发送	邀请记录创建
接受邀请	邀请者接受邀请	创建行程记录
拒绝邀请	邀请者拒绝邀请	状态更新为已拒绝

7 个人工作总结

7.1 小组分工

- **李盛鹏 2351136 (50%)**: 负责 flink 集群搭建, 告警系统设计与实现, 拼车模块实现。
- **程亦诚 2351582 (50%)**: 前后端搭建与部署, 交通数据采集与展示, 用户模块实现。

8 数据采集与集成的国内外现状

8.1 国外发展现状

8.1.1 主要平台与技术

Google Maps Platform 作为全球领先的地图服务平台, 其数据来源广泛, 包括卫星图像、市政合作伙伴数据以及用户众包信息, 能够实现每 2 至 5 分钟的实时更新频率, 覆盖全球 200 多个国家和地区。该平台的核心技术特点体现在 AI 驱动的路况预测、多源数据融合 (历史数据、实时数据与天气信息) 以及通过机器学习算法优化路线规划等方面, 为用户提供精准的交通信息服务。

HERE Technologies 则专注于高精度地图与自动驾驶领域, 其数据主要来源于专有车辆采集和移动设备众包。该平台提供高精度地图 (HD Map) 和自动驾驶级别的路况数据, 并通过开放数据

平台促进生态系统发展，成为智能交通领域的重要技术提供者。

TomTom Traffic 凭借其 20 多年的历史数据库积累，在路况数据服务领域具有独特优势。其数据来源包括 GPS 探针、移动设备和路侧传感器，技术特点集中在精准的 ETA 预测和路网拓扑分析，为交通规划和出行服务提供可靠支持。

8.1.2 技术趋势

当前国际上数据采集与集成技术呈现出明显的发展趋势。多源数据融合已成为主流方向，通过结合 GPS、摄像头、雷达、传感器等多模态数据，实现更全面的交通态势感知。AI 与机器学习技术广泛应用于交通流预测和异常检测，提升数据处理的智能化水平。边缘计算技术的推广使路侧单元能够实时处理数据，有效减少传输延迟。5G 与 V2X（车路协同）技术的发展促进了实时路况信息的共享与交互，而 Waze 模式的众包数据则为交通信息的实时性提供了有力保障。

8.2 国内发展现状

8.2.1 主要平台

百度地图在国内市场占据约 35% 的份额，位居行业第一。其数据来源多元化，包括自有数据采集车队、与交管部门的合作数据以及用户众包信息。百度地图的核心技术优势在于 AI 大脑（交通大脑）的应用，能够实现城市级交通信号优化，并提供智慧交管解决方案，为城市交通管理提供全方位支持。

高德地图以约 33% 的市场份额排名第二，依托阿里巴巴生态系统获取丰富数据资源，并与交管部门保持密切合作，同时通过用户众包不断补充实时信息。高德地图的技术特点体现在高德交通大脑的构建、动态路况预测能力以及智能调度系统，为用户提供精准的出行服务。

腾讯地图凭借约 20% 的市场份额位列第三，充分利用腾讯生态数据（如微信、QQ 等社交平台数据）和合作伙伴资源。其技术特色集中在 LBS 定位服务的精准性和社交化出行功能的创新，通过社交元素提升用户出行体验。

8.2.2 政策推动

国内智能交通领域的发展受到多项政策的有力推动。《交通强国建设纲要》明确提出到 2035 年实现智能交通世界领先的目标，为行业发展指明了方向。新基建政策重点支持 5G、北斗导航、车路协同等新一代信息技术在交通领域的应用，加速智能交通基础设施建设。智慧城市试点项目已在全国百余城市开展，推动智能交通技术的落地与实践。同时，数据开放政策的逐步推进使部分城市开始开放交通数据 API，为数据集成与创新应用提供了政策支持。

8.3 存在的问题

8.3.1 技术层面

数据孤岛问题是当前面临的主要技术挑战之一，各平台之间的数据难以互通，政府数据与企业数据未能充分融合，同时缺乏统一的数据标准，严重制约了数据集成的效率和效果。数据质量参差不齐也是突出问题，众包数据的准确性难以保证，部分地区的数据覆盖不足，而数据清洗和验证的高成本进一步增加了数据集成的难度。基础设施限制同样不容忽视，路侧传感器覆盖不足、5G 网络覆盖不均衡以及边缘计算节点不足等问题，影响了实时数据处理和传输的能力。

8.3.2 商业层面

商业层面的问题主要体现在 API 成本高昂和数据垄断两个方面。百度、高德等平台的 API 商用费用较高，免费额度限制严格，使得中小企业难以承担数据获取成本。同时，头部平台掌握大量交通数据资源，形成数据垄断局面，新进入者难以获取必要的的数据资源，而缺乏成熟的数据交易平台进一步加剧了这一问题，限制了市场的公平竞争和创新活力。

8.3.3 法律与隐私

随着数据安全意识的提升，法律与隐私问题日益凸显。根据数据安全法，位置数据属于敏感个人信息，需要严格的保护措施。对于国际平台而言，跨境数据流动需遵守 GDPR 等相关法规，增加了数据处理的复杂性。同时，数据脱匿名化的要求不断提高，需要更严格的技术手段和管理措施来保护用户隐私，这对数据采集与集成提出了更高的合规要求。

9 不足与努力方向

9.1 我们的不足

9.1.1 技术层面

A. 数据采集

数据采集方面存在多处不足：系统完全依赖百度、高德等商业 API，自主性较差；采集频率固定为 5 分钟间隔，无法根据不同时段的交通需求动态调整；同时缺少对 API 返回数据的异常检测机制。这些问题的影响是数据可靠性受第三方制约，API 限流可能导致采集失败，影响系统的数据完整性。

B. 后端服务

后端服务的不足体现在：缺少缓存机制，频繁直接查询数据库，未使用 Redis 等缓存技术优化性能；无异步处理机制，数据采集、邮件发送等耗时操作均同步执行；日志管理简陋，仅使用控制台输出日志，未集成 ELK 等日志分析平台；缺少系统性能监控和异常告警机制。这些问题在高并

发场景下会导致明显的性能瓶颈，影响用户体验。

C. 前端展示

前端展示方面的不足主要包括：地图集成较为简单，仅嵌入了 AMap 地图，未进行深度开发；无实时更新功能，数据需要手动刷新，未使用 WebSocket 等技术实现数据推送；图表功能单一，仅提供折线图，缺少热力图、流向图等更直观的可视化方式；移动端体验一般，仅具备基础的响应式设计，未针对移动设备做专门优化。这些问题导致用户粘性较低，无法满足实时性要求较高的应用场景。

9.1.2 功能层面

A. 智能推荐

智能推荐功能方面，系统同样存在功能缺失：缺少基于实时路况的路线推荐功能，无法为用户提供最优出行路径；拼车匹配采用人工选择方式，未实现智能匹配算法，效率较低；缺少个性化推荐功能，无法根据用户历史行为和偏好提供定制化服务。

B. 数据分析

数据分析功能方面，系统存在以下缺失：缺少交通拥堵热点分析功能，无法识别城市中的拥堵高发区域；未实现拥堵模式挖掘功能，无法发现拥堵的周期性和趋势性规律；缺少拥堵原因分析功能，无法对拥堵的具体原因（如事故、施工、车流过大等）进行深入分析。

9.2 努力方向

9.2.1 发展规划

在**短期阶段**，可以重点进行技术优化（引入 Redis 缓存、完善索引、增加单元测试、实现 WebSocket 实时推送）和功能增强（多源数据融合优化、增强历史数据可视化、优化拼车匹配功能）。

在**中期**，聚焦智能化升级（开发基于 LSTM 的路况预测模型、实现智能路线推荐、构建用户画像系统）和工程化建设（容器化部署、完善 CI/CD 流程、建立监控告警系统）。

长期来看，将建设自主数据源（自建数据采集系统、数据融合平台）、推进产品化（开发移动应用、构建开放平台、探索商业模式），并探索 5G 与车路协同、数字孪生、区块链等技术前沿在交通领域的应用。

9.2.2 学习与成长方向

A. 技术深度

学习与成长方向的技术深度提升主要包括三个方面：首先，深入学习大数据处理技术，包括 Spark、Flink 等流处理框架，以及数据仓库设计原理和实践；其次，专注于 AI/ML 领域，学习深度学习框架的使用，研究时序预测模型和强化学习算法在交通领域的应用；最后，探索分布式系统技术，包括微服务架构设计、消息队列应用和分布式缓存优化等，提升系统架构设计能力。

B. 领域知识

学习与成长方向的领域知识拓展主要包括三个方面：首先，深入学习交通工程领域知识，包括交通流理论、道路通行能力手册和交通规划方法等，了解交通系统的基本原理和规律；其次，掌握地理信息系统（GIS）相关知识，包括 GIS 基础、空间数据分析和地图可视化技术，提升交通数据的空间分析能力；最后，学习数据安全与隐私保护技术，包括数据脱敏技术、联邦学习和差分隐私等，确保数据的安全使用和隐私保护。

10 心得体会

10.1 项目总结与感悟

10.1.1 全栈开发的优势

通过本次项目，我深刻体会到全栈开发的优势：首先，全栈开发提供了全局视角，使我能够理解从数据采集到前端展示的完整数据流，更好地把握系统整体架构；其次，全栈开发具有自主可控的特点，不依赖其他团队，能够快速进行迭代开发；第三，在遇到跨层问题时，全栈开发背景有助于快速定位问题根源，提高问题解决效率；最后，全栈开发能够接触多种技术栈，有效提升综合技术能力。

当然，全栈开发也面临着一些挑战：需要掌握的技术栈较多，学习成本较高；各层都需要进行优化，容易导致精力分散；单人开发模式也缺乏团队协作经验的积累。

10.1.2 数据驱动的重要性

在数据驱动方面，我深刻认识到数据质量决定服务质量：初期未对 API 返回数据进行验证导致异常数据入库，后期增加数据清洗逻辑后系统稳定性明显提升，这让我充分理解了“垃圾进，垃圾出”（GIGO）的深刻含义。同时，数据采集是一切分析和应用的基础：没有高质量数据，再好的算法也无济于事；尽管多源数据融合过程复杂，但能显著提升系统可靠性；而且数据积累的价值往往需要长期才能充分体现。

10.1.3 用户体验至上

在用户体验方面，响应式设计的必要性得到了充分验证：初期仅针对桌面端开发导致移动端体验较差，后期优化响应式布局后用户满意度显著提升，让我意识到当前移动端流量已超过桌面端。同时，实时性也是提升用户体验的关键因素：静态数据展示无法满足用户对时效性的需求，通过 WebSocket 实现数据推送能显著提升用户体验，因此计划在下一个版本中重点优化这一功能。

10.2 技术选型的思考

10.2.1 框架选择

在框架选择上，我对 Vue 3 与 React 进行了对比：选择 Vue 3 主要是因为其学习曲线平缓、文档完善且生态丰富；实际开发中，Composition API 确实提供了灵活的开发方式，Vite 的开发体验也极为出色；不过从未来考虑，大型项目中 React 生态（如 Next.js）更为成熟。在后端框架方面，对 Spring Boot 与 Node.js 进行了评估：选择 Spring Boot 是基于其企业级标准、成熟的生态以及团队熟悉度；实际体验中 Spring Boot 开发效率高，但启动速度较慢；未来会考虑在高并发 I/O 密集型场景中使用 Node.js。

10.2.2 数据库选择

在数据库选择上，我对比了 MySQL、PostgreSQL 和 MongoDB：选择 MySQL 主要考虑到项目需求适合关系型数据，其事务支持良好且云服务成熟；实际使用中 MySQL 基本满足需求，但 JSON 字段支持方面不如 PostgreSQL；未来考虑使用 PostgreSQL，特别是其 GIS 扩展（PostGIS）更适合地理数据处理。

10.3 持续学习的必要性

10.3.1 技术更新快

技术更新速度之快令人印象深刻，前端领域经历了从 Vue 2 到 Vue 3（Composition API）的演进，构建工具也从 Webpack 革新到 Vite，组件开发方式从 Options API 过渡到 Composition API 再到 Script Setup。后端领域同样变化迅速，Spring Boot 从 2.x 版本升级到 3.x 版本（基于 Jakarta EE），系统架构从单体架构向微服务架构转变，处理方式也从同步处理向响应式编程（如 WebFlux）发展。

10.3.2 学习方法

在学习方法上，我总结了四点经验：首先，官方文档优先，因为它最权威且内容最新；其次，实践驱动学习，边做边学才能真正掌握技术，避免纸上谈兵；第三，积极关注社区动态，包括 GitHub、Stack Overflow 和各类技术博客；最后，定期总结所学内容，通过写博客、做笔记和分享经验来巩固知识。

10.4 项目管理经验

10.4.1 需求管理

在需求管理方面，我吸取了重要教训：初期需求不明确导致频繁返工，后期采用敏捷方法，通过小步快跑的方式显著提升了开发效率。基于这些经验，我提出了改进方向：优先开发 MVP（最小

可行产品)，使用用户故事（User Story）清晰描述需求，采用迭代开发和持续交付的方式推进项目。

10.4.2 时间管理

在时间管理上，我遇到了一些问题：完美主义倾向导致追求细节而拖延进度，技术难点耗时也经常超出预期。针对这些问题，我采取了相应对策：设定明确的优先级，先完成核心功能再优化细节；对关键技术进行预研，提前验证可行性；使用番茄工作法提高工作效率。

10.4.3 能力提升

在能力提升方面，我认识到技术能力、软技能和业务能力的全面发展至关重要。技术能力需要兼顾深度、广度和学习能力：深度上要精通某个技术栈，广度上要了解相关技术，同时具备快速掌握新技术的学习能力。软技能包括：清晰表达技术方案的沟通能力，与产品、设计、测试等团队成员协作的能力，以及分析和解决复杂问题的能力。业务能力则要求理解业务需求，能够用技术赋能业务，并具备一定的商业思维。

10.5 课程收获和建议

10.5.1 课程收获

在整学期的学习中，我从最初对数据采集流程只有大致概念，逐渐理解了数据从实时接入、流式处理到融合应用的完整链条。课堂上老师系统讲解了流数据处理框架、API 调用规范以及多源数据的采集与融合技术，这些知识在项目实践中得到了充分应用：当我尝试同时接入多个实时数据源并设计流处理逻辑时，才真正体会到窗口计算、状态管理和数据一致性的挑战；也正因为经历了这些实践，我更加理解了流式架构设计对数据时效性与准确性的关键影响。

实验环节让我对工具链有了具体感受。早期我只会用简单的请求脚本抓数据，经过课堂训练后，开始学着搭建批处理管线，给请求加重试和熔断，并用日志指标来观察稳定性。调优成功的那一刻，日志曲线平滑了，延迟分布收敛了，我才真正理解“工程化”三个字的分量。

我还感受到协作与分享的力量。每次课后讨论，听到同学们如何规避 IP 封禁、如何做字段映射，都会触发新的思路。后来把自己的踩坑记录整理成文档，反而在复盘时发现流程中的冗余步骤，顺手做了优化。这种“讲给别人听”的过程，比单纯做题更能加深理解。

10.5.2 课程建议

针对课程设置，我有几条具体建议。首先，理论与实践的节奏可以再紧凑一些：在讲完采集策略、数据清洗、数据融合等概念后，立即安排一个小型实操，让大家现场把策略落到真实 API 上，课堂上就能暴露问题并得到反馈。其次，希望增加对流处理和实时监控的配套讲解，这部分的入门资料不多，如果说能够在课上手把手讲解，或许大家的接受程度更好一些。再次，案例选择可更贴近行业场景：例如展示地图、金融、物联网等不同类型数据源的采集差异，让同学们理解“接口文

档之外”的隐性成本。最后，鼓励更多课堂内的同行评审和分享环节，让不同组的采集策略互相借鉴；如果能开放一两个优秀作业的代码讲解，会极大提升大家对工程化落地的直观认识。

10.6 总结

本项目成功实现了一个集实时路况监控与拼车服务于一体的综合平台，涵盖了数据采集、后端服务、前端展示三大核心模块。通过多源 API 集成实现了路况数据采集，使用 Spring Boot 构建了 RESTful API 服务，采用 Vue 3 实现了友好的用户界面，并结合 MySQL 数据库实现了数据持久化。

项目的主要成果包括：实现了支持百度、高德、腾讯三大地图 API 的多源数据采集功能；提供了城市级路况监控与统计的实时路况展示；支持时间范围查询与可视化的历史数据查询功能；构建了集用户认证、需求发布、邀请匹配于一体的拼车服务平台；以及采用响应式设计、支持多终端访问的用户友好界面。

通过本项目，我深入理解了全栈开发的完整流程，掌握了数据采集与集成的关键技术，积累了系统设计与实现的经验。同时，也认识到项目在技术深度、功能完整性、工程化水平等方面仍有较大提升空间。

未来，我将在数据质量优化、智能化升级、工程化建设等方面持续改进，努力将本项目打造成一个真正可用的产品，为智能交通系统的发展贡献自己的力量。